CSED233 · 데이터구조 · 포스텍

Ch5. Trees (1) — 기본 개념 & Binary Tree

1. Tree(트리) 기본 정의

이론

- **Tree**: node(원) + edge(선) 집합
- **Root(루트)**: 특별히 지정된 최상위 노드 (관례상 뒤집어 그림)
- 한 edge는 **child ↔ parent** 두 노드를 연결
- 한 노드는 여러 edge에 연결 가능
- **Terminal(단말 노드) (leaf, external)**: 자식 없는 노드
- **Non-terminal(비단말 노드) (internal)**: 자식 있는 노드

Recursive Definition

- Base: 단일 node 자체가 tree
- Step: root node n + 기존 tree들을 subtree로 붙이면 새 tree

2. 가족 관계 & Path(경로)

관계 용어

용어	의미
Parent(부모)/Child(자식)	A가 C의 parent
Ancestor(조상)/Descendant(후손)	경로상 위/아래
Sibling(형제)	같은 부모의 자식들

Path(경로)

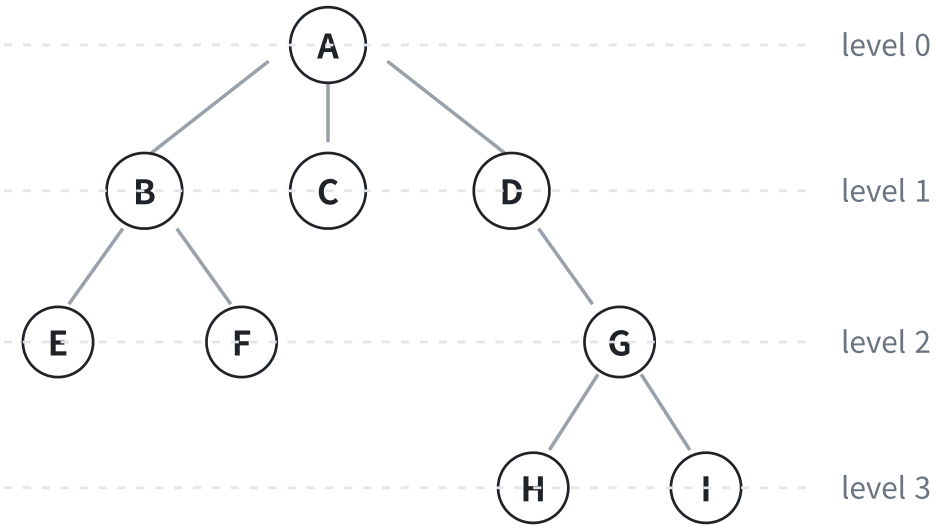
- **Path** $\langle n_1, n_2, \ldots, n_k \rangle$: n_i가 n_{i+1}의 parent
- **Length** = edge 수 = $k - 1$
- 예: A→B→E→H path는 length 3

3. Depth(깊이), Height(높이), Level

정의

개념	정의
Depth(깊이) of M	root → M 까지 path length (= M의 ancestor 수, 자신 제외)
Height(높이) of M	M → 가장 먼 leaf 까지의 path length
Height(높이) of Tree(트리)	height of root
Level	root = level 0, 자식 = level 1, ...

예시



root = level 0, 자식은 부모 level + 1 · Depth of M = 루트→M 경로 길이, Height = 가장 먼 리프까지의 경로 길이

- Depth(G) = 2, Height(G) = 1

- Height(F) = 0 (leaf)
- Height(T) = 3

4. Degree(차수) & Ordered Tree(트리)

- **Degree of node** = 자식 수 (leaf → 0)
- **Degree of tree** = max degree among all nodes
- **Ordered tree(순서 트리)**: sibling 간 linear ordering 존재 → 순서가 다르면 다른 tree
- **Forest(포레스트)**: tree들의 집합

5. Binary Tree(이진 트리)

정의

- **Binary tree**: 모든 노드가 **최대 2개** 자식. 각 child는 **left child** 또는 **right child** 로 designate됨.
- → 왼/오 구분이 있으므로 "L만 있는 tree" ≠ "R만 있는 tree"
- Binary tree는 **ordered tree**이다 (L/R 순서 고정).

특수 Binary Tree(이진 트리) 종류

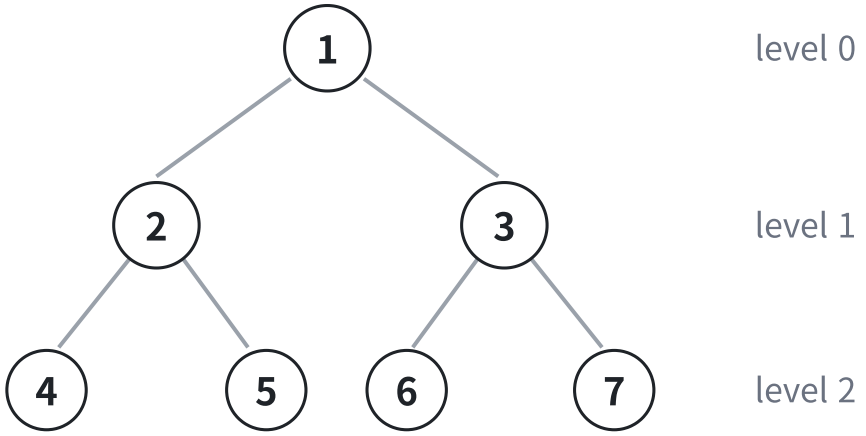
Proper (full 정의 1): 모든 노드가 자식 **0개 또는 정확히 2개 Full (full 정의 2)**: **모든 level이 꽉 참**

- height h인 full binary tree의 노드 수 = $2^{h+1} - 1$
- **유도**: level 0에 1개, level 1에 2개, ..., level h에 2^h개 → 합
  $= 1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1$ (기하급수)

6. Node Numbering (Full Binary Tree(포화 이진 트리))

번호 매기기

- 번호 1 (root) ~ $2^{h+1} - 1$
- Level 순 (위→아래), level 내 좌→우



배열 numbering: level 순(위→아래), level 내 좌→우 — 노드 k는 array[k]

번호로 계산되는 성질

연산	공식
Parent(부모) of k	$\lfloor k/2 \rfloor$ (if $k \neq 1$)
Left child of k	$2k$ (if $2k \leq n$)
Right child of k	$2k+1$ (if 2k+1 ≤ n)
Left sibling of k	$k-1$ (if k is odd, k>1)
Right sibling of k	$k+1$ (if k is even)

- → **이 관계 덕분에 array로 tree 저장 가능** (pointer 없이)

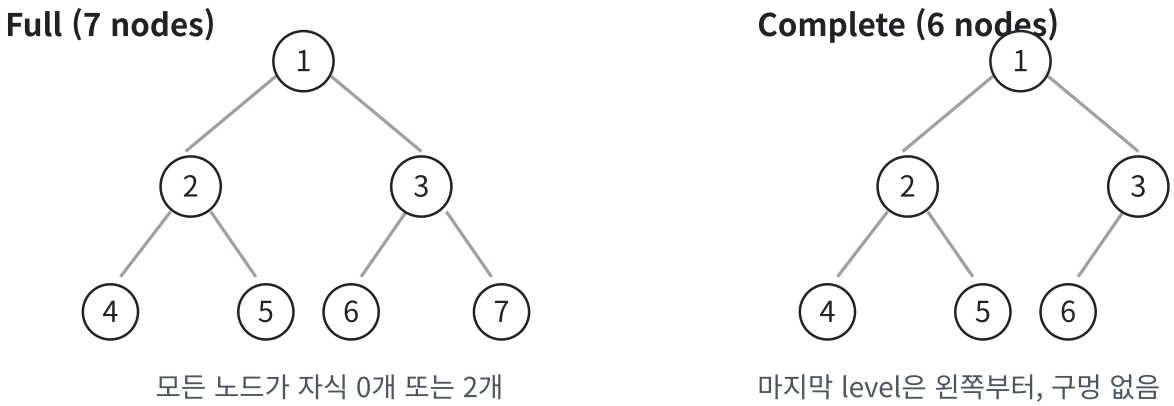
7. Complete Binary Tree(완전 이진 트리) (CBT)

정의

- **Relaxed full binary tree**: height h tree가 complete ⟺
  - Level $0 \sim h-1$ 은 전부 꽉 참
  - Level h (leaf level) 은 **왼쪽부터** 채워짐 (구멍 없음, 오른쪽만 빌 수 있음)

- → full은 complete이지만, 역은 아님

시각화



모든 노드가 자식 0개 또는 2개

마지막 level은 왼쪽부터, 구멍 없음

Full은 Complete이지만, Complete가 Full은 아님

→ Complete는 마지막 level이 "왼쪽부터 연속 채움". 중요 사용처: Heap(힙) (다음 Ch7) 구현은 CBT → array 표현 가능.

---

8. Binary Tree(이진 트리) 구현

Array(배열) 구현

- Full binary tree의 numbering을 그대로 사용
- char array[123]; — 노드 k는 array[k]에 저장
- **장점**: pointer 불필요, cache friendly
- **단점**: sparse tree면 공간 낭비 심함

Linked 구현

```
struct node {
  char val;
  node* left;
  node* right;
};
```

- 각 node에 2 pointer
- sparse/dynamic tree에 유리, 일반적

9. Tree(트리) Traversal (순회)

개념

- **Traversal(순회)**: 모든 노드를 **정확히 한 번** 체계적 방문
- 방문 시 action 수행: print, update, check, evaluate 등
- → tree를 **linearize** 하는 방법

DFS (Depth-First Search)

- 깊이 먼저 내려감
- 재귀 (또는 call stack) 기반
- 3가지 variation: **preorder / inorder / postorder**
- Time complexity(시간복잡도): $O(n)$ (모든 노드 1번씩 방문)

10. 세 가지 DFS Traversal(순회)

정의

- **Preorder(전위 순회)**: 노드 → 왼쪽 → 오른쪽

$$\text{Preorder}(T) = \langle n, \text{Preorder}(T_1), \ldots, \text{Preorder}(T_k) \rangle$$

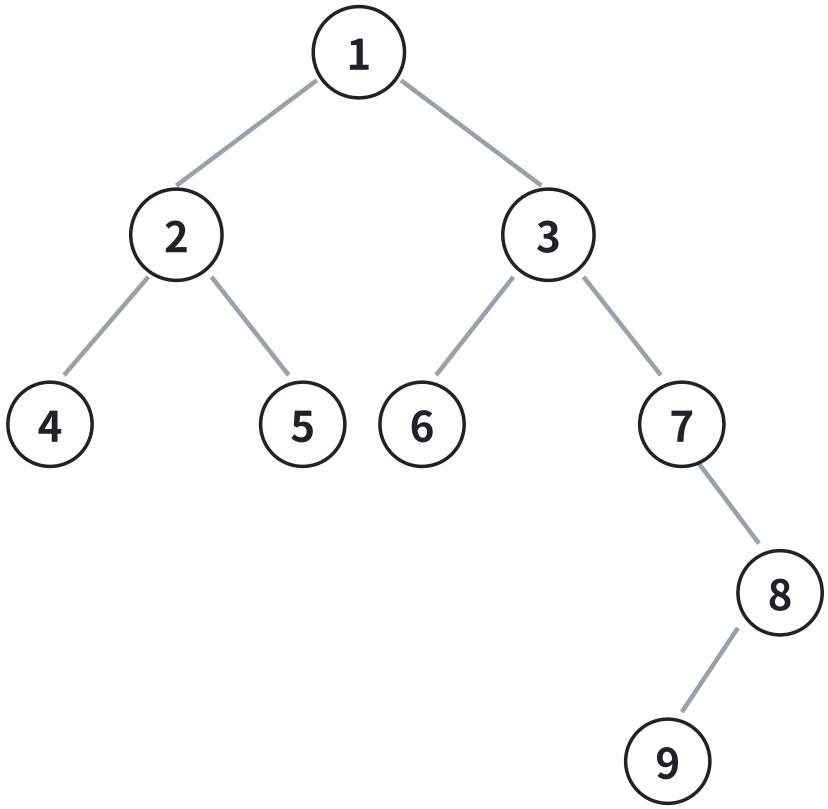
- **Inorder(중위 순회)**: 왼쪽 → 노드 → 오른쪽

$$\text{Inorder}(T) = \langle \text{Inorder}(T_1), n, \text{Inorder}(T_2), \ldots \rangle$$

- **Postorder(후위 순회)**: 왼쪽 → 오른쪽 → 노드

$$\text{Postorder}(T) = \langle \text{Postorder}(T_1), \ldots, \text{Postorder}(T_k), n \rangle$$

예제 Tree(트리)



예제 트리 — 7의 오른쪽 자식 8, 8의 왼쪽 자식 9

- **Preorder**: 1 2 4 5 3 6 7 8 9
- **Postorder**: 4 5 2 6 9 8 7 3 1
- **Inorder**: 4 2 5 1 6 3 9 8 7

Summary

- Tree 기본 용어 (root, parent, child, sibling, ancestor, descendant)
- Path, Depth, Height, Level, Degree
- Binary Tree (proper, full, complete)
- Node numbering & array 저장
- Traversal (preorder / inorder / postorder) — 모두 $O(n)$